

Java - Funktionale Programmierung



Dozent: Prof. Dr. Michael Eichberg
Kontakt: michael.eichberg@dhbw.de, Raum 149B
Version: 1.0.2

Teile der Folien basieren auf: Th. Letschert - Funktionale Programmierung in Java.

Folien: <https://delors.github.io/prog-adv-java-funktionale-programmierung/folien.de.rst.html>
<https://delors.github.io/prog-adv-java-funktionale-programmierung/folien.de.rst.html.pdf>

Kontrollfragen: <https://delors.github.io/prog-adv-java-funktionale-programmierung/kontrollfragen.de.rst.html>

Fehler melden: <https://github.com/Delors/delors.github.io/issues>

1. Einführung in die Funktionale Programmierung

Grundlagen funktionaler Programmierung

- Programmierparadigma, bei dem Funktionen im Mittelpunkt stehen
- Vermeidet veränderliche Zustände (🚫 *Mutable State*)
- Fördert deklarativen Code statt imperativem Code

? Frage

Wie unterscheidet sich dieses Paradigma von der objektorientierten Programmierung?

✓ Antwort

- Methoden ohne Seiteneffekte
- Daten sind standardmäßig unveränderlich
- Fokus auf Funktionsanwendungen und -komposition

Wichtige Konzepte

- Funktionen Höherer Ordnung
- Lambda-Ausdrücke
- Funktionskomposition
- Currying und Partielle Anwendung

2. Funktionale Programmierung in Java

Lambdas

Lambda (auch Closure):

Ein Ausdruck, dessen Wert eine Funktion ist.

Solche Ausdrücke sind sehr nützlich, mussten in Java aber vor Version 8 mit anonymen inneren Klassen emuliert werden.

Ein einfache Personenklasse

```
1 class Person {
2     private String name;
3     private int age;
4
5     public Person(String name, int age) {
6         this.name = name;
7         this.age = age;
8     }
9     public String getName() { return name; }
10    public int getAge() { return age; }
11    public String toString() { return "Person[" + name + ", " + age + "]; }
12 }
```

Sortieren von Personen nach Alter

Angenommen wir haben eine Klasse Person und eine Liste von Personen, die nach Alter sortiert werden soll. Dazu muss eine Vergleichsfunktion übergeben werden. In Java <8 kommt dazu nur ein Objekt in Frage.

```
1 List<Person> persons = Arrays.asList(
2     new Person("Hugo", 55),
3     new Person("Amalie", 15),
4     new Person("Anelise", 32) );
```

Traditionelle Lösung

```
1 Collections.sort(persons, new Comparator<Person>() {
2     public int compare(Person p1, Person p2) {
3         return p1.getAge() - p2.getAge();
4     }
5 });
```

Lösung ab Java 8+

```
1 Collections.sort(
2     persons,
3     (p1, p2) -> { return p1.getAge() - p2.getAge(); }
4 );
```

Lösung ab Java 8 (kürzer)

```
1 Collections.sort(persons, (p1, p2) -> p1.getAge() - p2.getAge());
```

▲ Achtung!

Bis Java 7 ist `java.lang.Object` der Basistyp aller Referenztypen. Der Typ eines Lambdas ist jedoch der Typ eines funktionalen Interfaces, das nur eine Methode hat und dieser

Typ muss explizit angegeben werden.

Instanzen von inneren Klassen können immer Object zugewiesen werden:

```
1 Object actionListener = new ActionListener() {  
2     @Override  
3     public void actionPerformed(ActionEvent e) {  
4         System.out.println(text);  
5     }  
6 };
```

Illegale Zuweisung:

```
1 Object actionListener = (e) -> System.out.println(text);  
2  
3 // Error: The target type of this expression must be a functional interface
```

Zuweisung an ein funktionales Interface:

```
1 ActionListener actionListener = (e) -> System.out.println(text);
```

Funktionale Interfaces

Functional Interface / SAM-Interface (Single Abstract Method Interface):

Ein Functional Interface ist ein Interface das genau eine Methode enthält (die natürlich abstrakt ist) optional kann die Annotation `@FunctionalInterface` hinzugefügt werden.

Beispiel

```
1 @FunctionalInterface  
2 interface MyActionListener extends java.awt.event.ActionListener {  
3     /*final static*/ int MAGIC_NUMBER = 42;  
4 }  
5  
6 MyActionListener actionListener =  
7     (e) -> System.out.println(text + MyActionListener.MAGIC_NUMBER);
```

Vordefinierte Funktionsinterfaces

`java.util.function` enthält viele vordefinierte Funktionsinterfaces, die in der funktionalen Programmierung häufig verwendet werden.

Beispiele sind:

- `Function<T,R>`: Eine Funktion, die ein Argument vom Typ `T` entgegennimmt und ein Ergebnis vom Typ `R` zurückgibt.
- `Predicate<T>`: Eine Funktion, die ein Argument vom Typ `T` entgegennimmt und ein Ergebnis vom Typ `boolean` zurückgibt.
- `Consumer<T>`: Eine Funktion, die ein Argument vom Typ `T` entgegennimmt und kein Ergebnis zurückgibt.
- `Supplier<T>`: Eine Funktion, die kein Argument entgegennimmt und ein Ergebnis vom Typ `T` zurückgibt.

Beispiel

Predicate<T>

```
1 static <T> List<T> filterList(List<T> l, Predicate<T> pred) {
2     List<T> res = new LinkedList<>();
3     for (T x : l) {
4         if (pred.test(x)) { res.add(x); }
5     }
6     return res;
7 }
8
9 List<Integer> l = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9);
10 System.out.println(filterList(l, x -> x % 2 == 0));
```

Ausgabe: [2, 4, 6, 8]

Beispiel

Consumer<T>

```
1 class WorkerOnList<T> implements Consumer<List<T>> {
2     private Consumer<T> action;
3     public WorkerOnList(Consumer<T> action) { this.action = action; }
4
5     @Override public void accept(List<T> l) {
6         for (T x : l) action.accept(x);
7     }
8
9     WorkerOnList<Integer> worker =
10         new WorkerOnList<>((i) -> System.out.println(i*10));
11     worker.accept(Arrays.asList(1,2,3,4));
```

Ausgabe: 10 20 30 40

Lambdas - Method References

Als Implementierung eines funktionalen Interfaces (als „Lambda“) können auch Methoden verwendet werden.

Beispiel

Referenz auf statische Methode

```
1 class ListMethods {
2     static <T> List<T> filterList(List<T> l, Predicate<T> pred) {
3         List<T> res = new LinkedList<>();
4         for (T x : l) if (pred.test(x)) { res.add(x); }
5         return res;
6     }
7     static boolean isEven(int x) { return x % 2 == 0; }
8 }
9
10 List<Integer> l = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9);
11 System.out.println(filterList(l, ListMethods::isEven));
```

Ausgabe: [2, 4, 6, 8]

Beispiel

Referenz auf Instanzmethode

```
1 class Tester {
2     private int magicNumber;
3     public Tester(int magicNumber) { this.magicNumber = magicNumber; }
4     boolean isMagic(int x) { return x == magicNumber; }
5 }
6 class ListMethods {
7     static <T> List<T> filterList(List<T> l, Predicate<T> pred) {
8         List<T> res = new LinkedList<>();
9         for (T x : l) if (pred.test(x)) res.add(x);
10        return res;
11    }
12 }
13 List<Integer> l = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9);
14 System.out.println(filterList(l, new Tester(5)::isMagic));
```

Beispiel

Referenz auf Konstruktor

```
1 class Tester {
2     private int magicNumber;
3     public Tester(int magicNumber) { this.magicNumber = magicNumber; }
4     boolean isMagic(int x) { return x == magicNumber; }
5 }
6
7 Function<Integer, Tester> create = Tester::new;
8 create.apply(5).isMagic(5);
```

Ausgabe: true

Erweiterungen der Collection API

Neue Methoden in der Collection API

- `forEach(Consumer<? super T> action)`
- `removeIf(Predicate<? super T> filter)`
- `replaceAll(UnaryOperator<T> operator)`
- `sort(Comparator<? super T> c)`

Beispiel

```
1 List<Integer> l = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9);  
2 l.replaceAll(x -> x * 10);  
3 l.forEach(System.out::println);
```

Ausgabe: 10 20 30 40 50 60 70 80 90

Übung

2.1. Erste Implementierung von Funktionen höherer Ordnung

Schreiben Sie eine Klasse `Tuple2<T>`; d. h. eine Variante von `Pair` bei der beide Werte vom gleichen Typ `T` sein müssen. Die Klasse soll Methoden haben, um die beiden Werte zu setzen und zu lesen und weiterhin um folgende Methoden ergänzt werden:

- `void forEach(Consumer<...> action)`: Führt die Aktion für jedes Element in der `Queue` aus.
- `void replaceAll(UnaryOperator<...> operator)`: Ersetzt alle Elemente in der `Queue` durch das Ergebnis der Anwendung des Operators auf das Element.

Schreiben Sie Tests für die neuen Methoden; verwenden Sie dafür Closures bzw. Lambda-Funktionen. Stellen Sie 100% *Statementcoverage* sicher.

Die Java Dokumentation finden Sie hier:

- Übersicht: <https://docs.oracle.com/en/java/javase/24/docs/api/help-doc.html>
- API Dokumentation: <https://docs.oracle.com/en/java/javase/24/docs/api/allclasses-index.html>
(Hier der Link auf die Dokumentation der Klasse `UnaryOperator`: <https://docs.oracle.com/en/java/javase/24/docs/api/java.base/java/util/function/UnaryOperator.html>)

⌘ Hinweis

- Sorgen Sie ggf. vorher dafür, dass Sie eine angemessene Projektstruktur haben.
- Passen Sie ggf. die `pom.xml` von ihren anderen Projekten an.

Warteschlangen

Eine Warteschlange ist eine einfache Datenstruktur bei der die Elemente in der Reihenfolge entfernt werden in der diese hinzugefügt wurden (📁 *First-in-First-out (FiFo)*).

Zentrale Methoden einer Warteschlange `Queue<T>` sind:

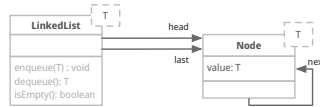
- `void enqueue(T item)` (auch `boolean offer(T item)` bzw. `void add(T item)`): Fügt ein Element hinzu.
- `void dequeue(T item)` oder `T poll()`: Entfernt das älteste Element und gibt es zurück.
- `boolean isEmpty()`: Gibt an ob die Warteschlange leer ist.

Verkettete Listen

Bemerkung

Bisher haben wir Arrays verwendet, um Listen zu speichern. Arrays haben jedoch den Nachteil, dass sie eine feste Größe haben und nicht effizient vergrößert werden können.

Eine verkettete Liste ist eine alternative Implementierungstechnik, die flexibler ist als ein Array und dynamisch wachsen kann. Jedes Element in einer verketteten Liste enthält eine Referenz auf das nächste Element in der Liste.



Übung

2.2. Implementierung einer Warteschlange mittels verketteter Liste

Implementieren Sie eine Warteschlange (`Queue<T>`) basierend auf einer verketteten Liste. Ihre Klasse `LinkedListQueue<T>` soll das folgende Interface `Queue<T>` implementieren.

```
1 public interface Queue<T> {  
2     void enqueue(T item);  
3     T dequeue();  
4     boolean isEmpty();  
5     int size();  
6  
7     void replaceAll(UnaryOperator<T> operator);  
8     void forEach(Consumer<T> operator);  
9     <X> Queue<X> map(Function<T, X> mapper);  
10    static <T> Queue<T> empty() { TODO }  
11 }
```

Erklärungen

map: Erzeugt eine neue `Queue<X>` bei der die Elemente der neuen Queue das Ergebnis der Anwendung der Funktion `apply` des Objekts `mappers` auf die Element der `Queue` sind.

empty: Erzeugt eine leere Queue.

Schreiben Sie Testfälle, um die Implementierung zu überprüfen. Zielen Sie auf mind. 100% *Statementcoverage* ab.

Zusammenfassung

Funktionale Programmierung

- Lambdas sind Ausdrücke, die (anonyme) Funktionen repräsentieren.
- Es sind sowohl Referenzen auf statische Methoden als auch Instanzmethoden und sogar Konstruktoren möglich.
- Currying wird nicht direkt unterstützt, aber durch die Verwendung von Funktionskomposition und partieller Anwendung kann es simuliert werden.

Warteschlangen

- Queues realisieren das Konzept einer Warteschlange bei der die Elemente, die zuerst hinzugefügt wurden, auch zuerst wieder entfernt werden (FiFo).
- Eine Implementierungsstrategie für Queues ist die Verwendung einer verketteten Liste.

3. Java Streams

Streams - Einführung

Streams sind umgeformte Sammlungen, die durch die Umformung für funktional-orientierte Massen-Operationen geeignet sind.

Beispiel

```
1 import java.util.Arrays;
2 import java.util.List;
3 import java.util.stream.Collectors;
4
5 List<Integer> l = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9);
6 List<Integer> ll = l
7     .stream() // list → stream
8     .filter(x → x % 2 == 0) // filter list with predicate
9     .map(x → 10 * x) // map each element to a new one
10    .collect(Collectors.toList()); // back to a list
11 ll.forEach(x → System.out.println(x));
```

Ausgabe: 20 40 60 80

Streams mit primitiven Daten und Objekten

- `Stream<T>` ist der Typ der Streams mit Objekten vom Typ `T`
- Streams mit primitiven Daten:

- `IntStream`
- `LongStream`
- `DoubleStream`

Dies Streams mit primitiven Daten arbeiten in vielen Fällen effizienter jedoch sind manche Operationen nur auf `Object`-Streams erlaubt. „Primitive“ Streams können mit der Methode `boxed` in `Object`-Streams umgewandelt werden.



Beispiel

```
1 IntStream isPrim = IntStream.range(1, 10);  
2 Stream<Integer> isObj = isPrim.boxed();
```

Erzeugung von Streams

Statische Methoden in `Arrays`

- Die Klasse `java.util.Arrays` hat mehrere überladene statische stream-Methoden, mit denen Arrays in Ströme umgewandelt werden können.
- Die Streams können Objekte oder primitive Daten enthalten.

Beispiel

```
1 // Stream of primitive data:
2 IntStream isP = Arrays.stream(new int[] { 1, 2, 3, 4, 5, 6, 7, 8, 9, 0 });
3 // Stream of objects:
4 Stream<Integer> isO = Arrays.stream(
5     new Integer[] { 1, 2, 3, 4, 5, 6, 7, 8, 9, 0 }
6 );
```

Statische Methoden in `Stream`

- Das Interface `java.util.stream.Stream` enthält mehrere statische Methoden mit denen Streams erzeugt werden können.
- Für die Klassen der Streams mit primitiven Werten (z.B. `java.util.stream.IntStream`) gibt es äquivalente Methoden.
- Mit `of` werden die übergebenen Wert in einen Stream gepackt.
- Mit `iterate` und `generate` hat man eine einfache Möglichkeit unendliche Ströme zu erzeugen.

Beispiel

```
1 // Object-Stream 1, 2 ... 9, 0:
2 Stream<Integer> is1a = Stream.of(1,2,3,4,5,6,7,8,9,0);

3 // int-Stream 1, 2, ... 9, 0
4 IntStream is1b = IntStream.of(1,2,3,4,5,6,7,8,9,0);

5 // (infinite) Stream 1, 2, ...
6 Stream<Integer> is2 = Stream.iterate(1, ((x) -> x+1));

7 int[] z = new int[]{1};
8 Stream<Integer> is3 = Stream.generate(() -> z[0]++); // (infinite) Stream 1, 2, ...
```

Statische range-Methoden in `IntStream` und `LongStream`

Die Interfaces `java.util.stream.IntStream` und `java.util.stream.LongStream` enthalten jeweils zwei statische `range`-Methoden mit denen Streams erzeugt werden können.

Beispiel

```
1 IntStream isPrimA = IntStream.range(1, 10); // 1,2, .. 9
2 IntStream isPrimA = IntStream.rangeClosed(1, 10); // 1,2, .. 9, 10
```

Nicht-statische Methoden der Collection-API

Das Interface `java.util.Collection` enthält die Methode `stream` mit der die jeweilige Kollektion in einen Stream umgewandelt werden kann.

Beispiel

```
1 Stream<Integer> is = Arrays.asList(1,2,3,4,5,6,7,8,9,0).stream();
```

Verwendung von Streams

Streams werden typischerweise in einer Pipeline-artigen Struktur genutzt:

1. Erzeugung
2. Folge von Verarbeitungs-/Transformationsschritten
3. Abschluss mit einer terminalen Operation

Verarbeitungsoperationen

Verarbeitungs-Operationen transformieren die Elemente eines Streams. Man unterscheidet:

■ zustandslose Operationen

Transformieren die Elemente jeweils völlig unabhängig von allen anderen.

■ zustandsbehaftete Operationen

Transformieren die Elemente abhängig von anderen.

Zustandslose Verarbeitungsoperationen

- `map(Function<? super T, ? extends R> mapper)`: Transformiert jedes Element in ein anderes.
- `filter(Predicate<? super T> predicate)`: Filtert Elemente heraus.
- `flatMap(Function<? super T, ? extends Stream<? extends R>> mapper)`: Transformiert jedes Element in einen Stream und fügt die Streams zusammen.

Beispiel

```
1 import java.util.List;
2 import java.util.stream.Collectors;
3 import java.util.stream.IntStream;
4
5 List<Integer> is = IntStream.range(1, 10)
6     .filter(i -> i % 2 != 0)
7     .peek(i -> System.out.print(i + " "))
8     .map(i -> 10 * i)
9     .boxed()
10    .collect(Collectors.toList());
11 System.out.println(is);
```

Ausgabe: 1 3 5 7 9 [10, 30, 50, 70, 90]

Beispiel

```
1 import java.util.List;
2 import java.util.stream.Collectors;
3 import java.util.stream.IntStream;
4 import java.util.stream.Stream;
5
6 static Stream<Integer> range(int from, int to) {
7     return IntStream.range(from, to).boxed();
8 }
9
10 List<Integer> is = Stream.of(0, 1, 2)
```

```
11 .flatMap(i → range(10 * i, 10 * i + 10))
12 .collect(Collectors.toList());
```

Ausgabe: is $\Rightarrow$ [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29]

Zustandsbehaftete Verarbeitungsoperationen

- `distinct()`: Entfernt Duplikate.
- `sorted()`: Sortiert die Elemente.
- `sorted(Comparator<? super T> comparator)`: Sortiert die Elemente mit einem gegebenen Comparator.
- `limit(long maxSize)`: Begrenzt die Anzahl der Elemente.
- `skip(long n)`: Überspringt die ersten n Elemente.

Beispiel

```
1 import java.util.List;
2 import java.util.stream.Collectors;
3 import java.util.stream.Stream;
4
5 List<Integer> lst = Stream.of(9, 0, 3, 1, 7, 3, 4, 7, 2, 8, 5, 0, 6, 2)
6     .distinct()
7     .sorted((i, j) → i - j)
8     .skip(1)
9     .limit(3)
10    .collect(Collectors.toList());
```

Ausgabe: is $\Rightarrow$ [1, 2, 3]

Verarbeitungsoperationen

Eine terminale Operation hat im Gegensatz zu den Verarbeitungsoperationen *keinen* *Stream* als Ergebnis.

Terminale Operationen ohne Ergebnis

- `forEach(Consumer<? super T> action)`
Wendet die übergebene Aktion auf alle Elemente des Streams an.

Terminale Operationen mit Ergebnis

- Operationen mit Array-Ergebnis: `Stream  $\Rightarrow$  Array`
Operationen die den Stream in ein äquivalentes Array umwandeln.
- Operationen mit Kollektions-Ergebnis: `Stream  $\Rightarrow$  Kollektion`
Operationen die den Stream in eine äquivalente Kollektion umwandeln.
- Operationen mit Einzel-Ergebnis: Aggregierende Operationen
Operationen die den Stream zu einem einzigen Wert verarbeiten.

Beispiel

`forEach`

```

1 Stream.of(9, 0, 3, 1, 7, 3, 4, 7, 2, 8, 5, 0, 6, 2)
2   .distinct()
3   .sorted( (i,j) → i-j )
4   .limit(3)
5   .forEach( System.out::println );

```

Beispiel

toArray

```

1 int[] a = IntStream.range(1, 3).toArray();
2
3 Object[] a = Stream.of("1", "2", "3").map( Integer::parseInt )
4   .toArray();
5
6 Integer[] a = (Integer[]) Stream.of(1, 2, 3)
7   .toArray();
8
9 String[] a = Stream.of(1, 2, 3).map( (i) → i.toString() )
10  .toArray( String[]::new ); // using generator

```

Terminale Operationen mit Kollektions-Ergebnis

- Die Methode `collect` erzeugt eine Kollektion aus den Elementen des Streams.
- `IntStream` und andere Streams mit primitiven Daten haben keine entsprechende Operation.
- Das Argument von `collect` ist ein `java.util.stream.Collector`. Die Erzeugung einer Kollektion ist damit Sonderfall einer aggregierenden Operation.
- Für die Erzeugung einer Kollektion verwendet man typischerweise einen vordefinierten `Collector` aus der Klasse `java.util.stream.Collectors`.
- Einfache Kollektionserzeuger in Collectors sind:

```

■ toList()
■ toSet()
■ toCollection(Supplier<C> collectionFactory)

```

Beispiel

collect

```

1 List<Integer> l1 = Stream.of(1, 2, 3).collect( Collectors.toList() );
2
3 List<Integer> l2 = IntStream.range(1, 4).boxed()
4   .collect( Collectors.toList() );
5
6 Set<String> s1 = (Set<String>) Stream.of("1", "2", "3")
7   .collect( Collectors.toSet());
8
9 Set<String> s2 = (Set<String>) Stream.of("1", "2", "3")
10  .collect( Collectors.toCollection( HashSet::new ) );
1
2 // Generating a map from a stream of strings

```

```

3 Map<String, Integer> m = Stream.of("1", "2", "3")
4     .collect(
5         Collectors.toMap(
6             (s) -> s,
7             Integer::parseInt
8         )
9     );

```

In `Collectors` finden sich **Kollektoren mit denen Maps erzeugt werden können**, die eine Gruppierung bzw. eine Partitionierung der Stream-Elemente darstellen:

- `static <T,K> Collector<T,?,Map<K,List<T>>> groupingBy(Function<? super T,? extends K> classifier)`

Gruppiert die Elemente entsprechend einer Klassifizierungsfunktion.

- `static <T> Collector<T,?,Map<Boolean,List<T>>> partitioningBy(Predicate<? super T> predicate)`

Partitioniert die Elemente entsprechend einem Prädikat.

Beispiel

`collect(groupingBy)`

Hilfreiche Methoden

```

1 import static java.util.stream.Collectors.groupingBy;
2 import static java.util.stream.Collectors.partitioningBy;
3 import static java.util.stream.Collectors.counting;

```

```

1 Map<Integer, List<Integer>> groupedByMod3 = Stream.of(1, 2, 3, 4, 5, 6, 7, 8, 9)
2     .collect( groupingBy( (x) -> x%3 ) );

```

Ausgabe: `groupedByMod3 = {0=[3, 6, 9], 1=[1, 4, 7], 2=[2, 5, 8]}`

```

1 Map<Integer, List<String>> groupedByLength = Stream.of(
2     "one", "two", "three", "four", "five", "six", "seven", "eight")
3     .collect( groupingBy( (s) -> s.length() ) );

```

Ausgabe: `groupedByLength ==> {3=[one, two, six], 4=[four, five], 5=[three, seven, eight]}`

Das Interface `Stream` bzw. die Interfaces für Ströme primitiver Daten (`IntStream`, etc.) bieten einige **einfache aggregierende Funktionen für Standardoperationen** auf allen Elementen des Stroms.

Beispiel

```

1 long count = Stream.of(1, 2, 3, 4, 5, 6, 7, 8, 9).count();
2
3 long sum = IntStream.of(1, 2, 3, 4, 5, 6, 7, 8, 9).sum();
4
5 OptionalDouble av = IntStream.of(1, 2, 3, 4, 5, 6, 7, 8, 9).average();

```

Das Interface `Stream` bieten einige einfache **aggregierende Funktionen für den Test aller Elemente des Stroms** mit einem übergebenen Prädikat.

Beispiel

```
1 boolean anyEven = Stream.of(1, 2, 3, 4, 5, 6, 7, 8, 9)
2   .anyMatch( (x) → x%2 == 0 );
3
4 boolean allEven = Stream.of(1, 2, 3, 4, 5, 6, 7, 8, 9)
5   .allMatch( (x) → x%2 == 0 );
6
7 boolean noneEven = Stream.of(1, 2, 3, 4, 5, 6, 7, 8, 9)
8   .noneMatch( (x) → x%2 == 0 );
```

Das Interface `Stream` bietet die Funktionen `findFirst` und `findAny` für die „Suche“ nach dem ersten bzw. irgendeinem Element in einem Stream.

⚠ Achtung!

Diese Methoden haben kein Prädikat als Parameter. Es empfiehlt sich darum den `Stream` vorher mit dem entsprechenden Prädikat zu filtern.

Beispiel

```
1 Optional<Integer> firstEven = Stream.of(1, 2, 3, 4, 5, 6, 7, 8, 9)
2   .filter( (x) → x%2 == 0 )
3   .findFirst();
```

Ausgabe: `firstEven` $\Rightarrow$ `Optional[2]`

Das Interface `Stream` bietet die Funktion

```
Optional<T> reduce(BinaryOperator<T> accumulator)
```

mit der eine Funktion auf jedes Element und das bisherige Zwischenergebnis angewendet werden kann.

Falls der erste Wert nicht der Startwert sein soll, verwendet man:

```
Optional<T> reduce(T identity, BinaryOperator<T> accumulator)
```

Beispiel

`reduce`

```
1 Optional<Integer> sumOfAll = Stream.of(1, 2, 3, 4, 5).reduce( (a, x) → a+x );
```

Ausgabe: `sumOfAll` $\Rightarrow$ `Optional[15]`

```
1 Optional<Integer> subOfAll = Stream.of(1, 2, 3, 4, 5).reduce( (a, x) → a-x );
```

Ausgabe: `subOfAll` $\Rightarrow$ `Optional[-13]`

```
1 int sumOfAllPlus100 = Stream.of(1, 2, 3, 4, 5)
2   .reduce(100, (a, x) → a+x );
```

Ausgabe: `sumOfAllPlus100` $\Rightarrow$ `115`

Es gibt einen Kollektor mit dem String-Elemente zu einem String konkateniert werden können:

```
static Collector<CharSequence,?,String> joining(CharSequence delimiter)
```

Beispiel

reduce

```
1 String concat = Stream.of("one", "two", "three")  
2   .collect( joining("+") );
```

Ausgabe: concat = one+two+three

Streams - fortgeschrittene Konzepte

Ausgewählte Eigenschaften des *Basisinterface* aller Streams

Parallele und sequentielle Streams.

```
1 package java.util.stream;
2
3 public interface BaseStream<T, S extends BaseStream<T,S>> {
4     /** Closes the stream, releasing any resources associated with it. */
5     void close();
6
7     /** Returns an equivalent stream that is parallel. */
8     S parallel();
9     /** Returns an equivalent stream that is sequential. */
10    S sequential();
11 }
12
13 public interface Stream<T> extends BaseStream<T, Stream<T>> {
14     // ...
15 }
```

Die Interfacedefinition (`BaseStream<T, S extends BaseStream<T,S>>`) ist eine Anwendung des CRTP; d. h. des *Curiously Recurring Template Patterns*. Bei diesem Idiom haben wir eine Klasse `X`, die von einer generischen Klasse oder einem generischen Interface `S` abgeleitet wird, wobei die ableitende Klasse `X` sich selber als Typparameter verwendet. Dies erlaubt die Definition einer Fluent-API, bei der Methoden, die in der Basisklasse definiert sind, den abgeleiteten Typ zurückgeben.

Erzeugen von eigenen Streams mittels `StreamSupport`

Die Implementierung des Interfaces `Stream<T>` ist ggf. sehr aufwändig. Alternativ kann die Klasse `StreamSupport` verwendet werden, um auf einem `Spliterator` basierende Streams zu erzeugen.

```
1 package java.util.stream;
2
3 public final class StreamSupport {
4
5     /** Creates a new sequential or parallel Stream from a Spliterator. */
6     static <T> Stream<T> stream(Spliterator<T> spliterator, boolean parallel);
7
8     // ...
9 }
```

Java Optionals

Instanzen der Klasse `java.util.Optional<T>` (bzw. `java.util.OptionalInt` etc.) **repräsentieren Werte die vorhanden sind oder auch nicht**.

Insbesondere `java.util.Optional<T>` kann/sollte anstelle von `null` verwendet werden, in Fällen in denen unter bestimmten Umständen kein sinnvoller Wert angegeben werden kann.

◆ Bemerkung

Es gibt moderne Programmiersprachen, die auf `null` komplett verzichten und stattdessen immer auf `Optionals` oder ähnliche Konstrukte setzen.

📄 Beispiel

```
1 static Optional<Integer> min(int[] a ) {  
2     if(a == null || a.length == 0)  
3         return Optional.empty();  
4  
5     int min = a[0];  
6     for(int x: a) { if (x < min) { min = x; } }  
7     return Optional.of(min);  
8 }
```

Übung

3.1. Java Streams

Bemerkung

Verwenden Sie ausschließlich Streams und Lambda-Ausdrücke.

1. Schreiben Sie eine Methode `int sumOfSquares(int[] a)` die die Elemente des Arrays quadriert und dann die Summe berechnet.
2. Schreiben Sie eine Methode `int sumOfSquaresEven(int[] a)` die die Elemente des Arrays quadriert, und dann die Summe berechnet für alle Elemente die gerade sind.
3. Schreiben Sie eine Methode, die eine Liste von Strings (`List<String>`) in eine flache Liste von Zeichen (`List<Integer>`) umwandelt.
4. Schreiben Sie eine Methode, die die Zahlen von 1 bis `Integer.MAX_VALUE` addiert. Nutzen Sie `IntStream.range()` um die Zahlen zu iterieren. Messen Sie die Ausführungsdauer für die *sequentielle* und *parallele* Ausführung (siehe Anhang für eine entsprechende Methode zur Zeitmessung.)

Um die Ausführungsdauer Ihrer Methode zu messen, können Sie folgenden Methode verwenden:

```
1 void time(Runnable r) {  
2     final var startTime = System.nanoTime();  
3     r.run();  
4     final var endTime = System.nanoTime();  
5     System.out.println("elapsed time: " + (endTime - startTime));  
6 }
```

Ein Aufruf der Methode `time` könnte dann so aussehen:

```
1 time(() -> System.out.println(sumOfSquares(new int[]{1,2,3,4,5,6,7,8,9,0})));
```